22/08/25

EXP:-

5. Study of Activation functions and its role

Aim:-

To study different activation function and understand their role.

Objective:-

1. To implement and visualize common activation functions such as sigmoid. Tanh. ReLU and softmax.
2. To understand the importance of activation function in transforming input symbols.
3. To observe how activation functions affect the training & performs of models.

Pseudocode:-

Start.

1. import necessary libraries (numpy, matplotlib, etc,)
2. Define activation function.
   (i) sigmoid (x) = 1/((1+e^(-x))
   (ii) Tanh (x) = (e^x - e^(-x))/(e^x + e^(-x))
   (iii) ReLU (x) = max (0, x)
   (iv) softmax(xi) = e^(xi) / Σe^(xj) for all j
3. Generate input value in a range (eg:- -10 to 10)
4. Apply each activation function on the input values.
5. plot the output of each function to visualize their behaviour.
6. Compare and observe.
   - Range of outputs

- Non-linearity introduced.
- suitability for classification / regression.

End.

Observation:-

1. Sigmoid: squashes values between 0 and 1. useful for probability but suffers from vanshing grandients.
2. Tanh:- Squashes value between -1 and 1. centred at zero better than sigmoid in some cases.
3. ReLU:- Output 0 for negative input and linear for positive avoids vanishing gradiants commonly used in hidden layer.
4. Softmax:- convert output into probabilities that sum to 1 mainly used in final layer of classification tasks.

Result:-

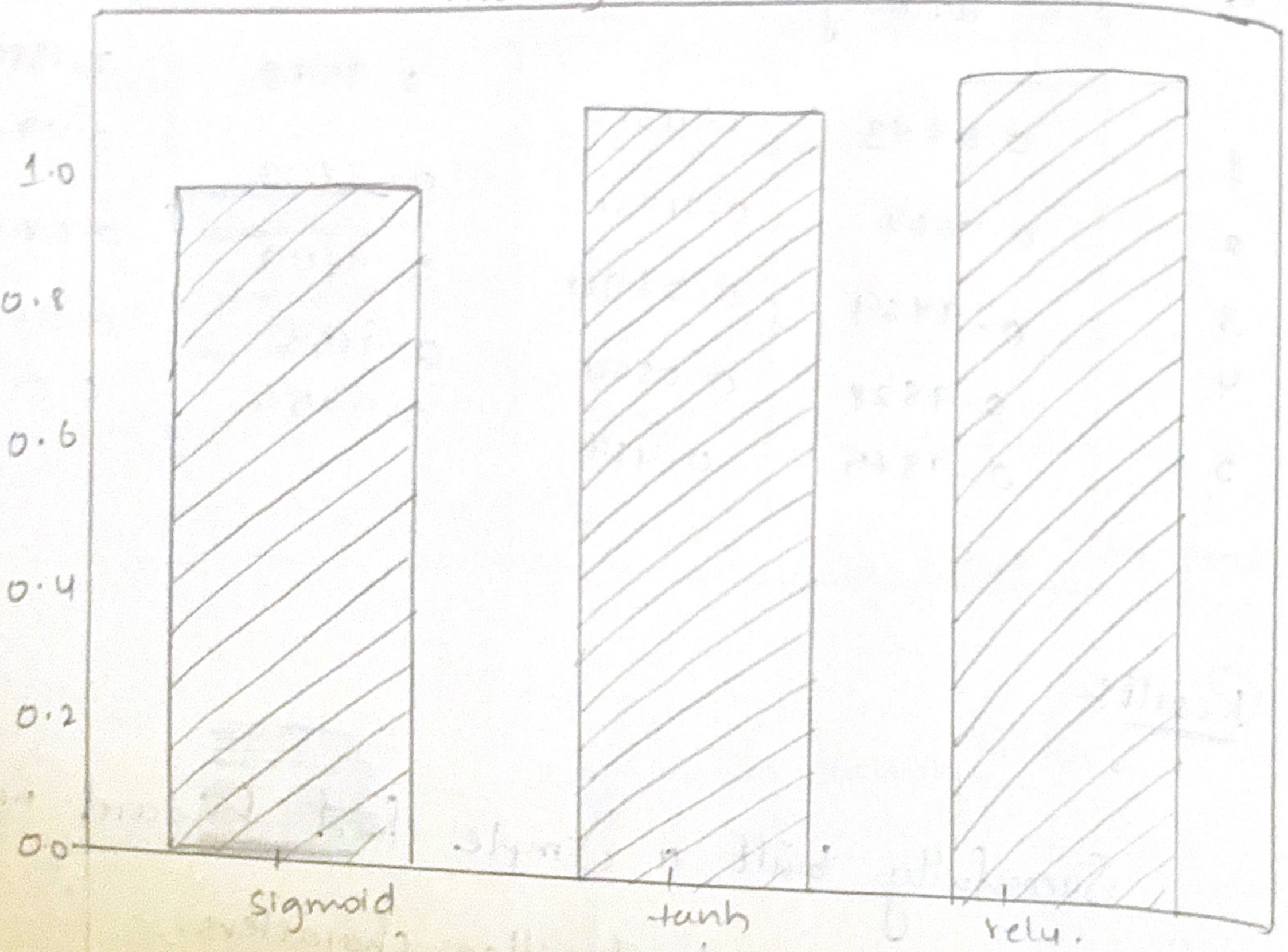
Sucessfully studied and implement activation functions and understood their role.

Training with Sigmoid. activation

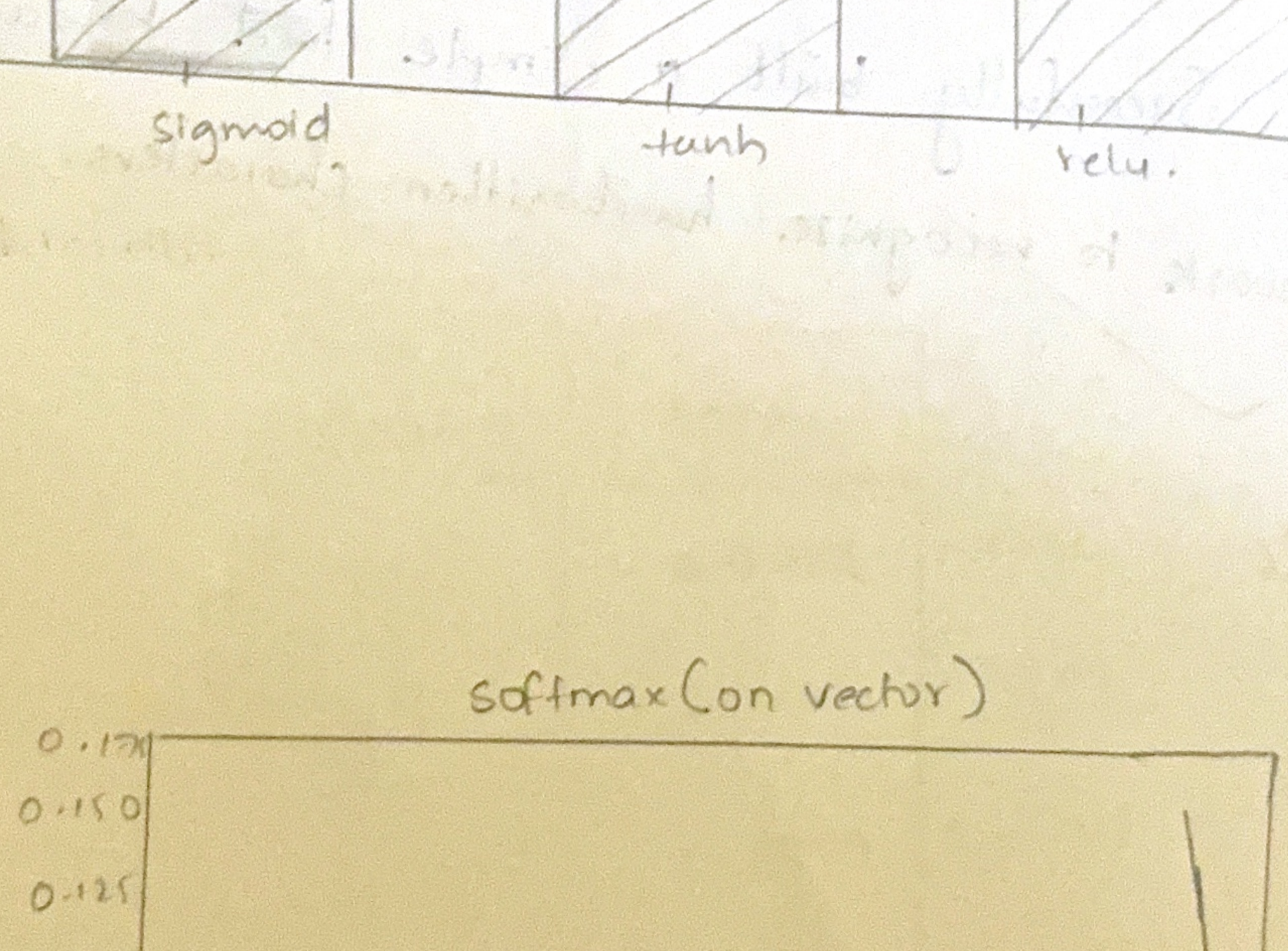
Training with tanh activation

Training with relu activation.

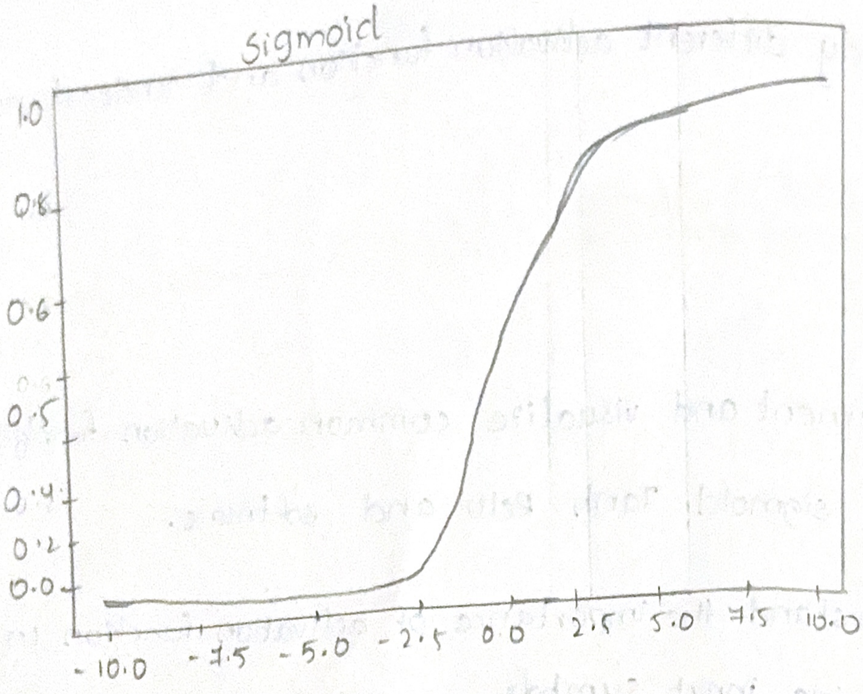
Accuracy with different Activation function



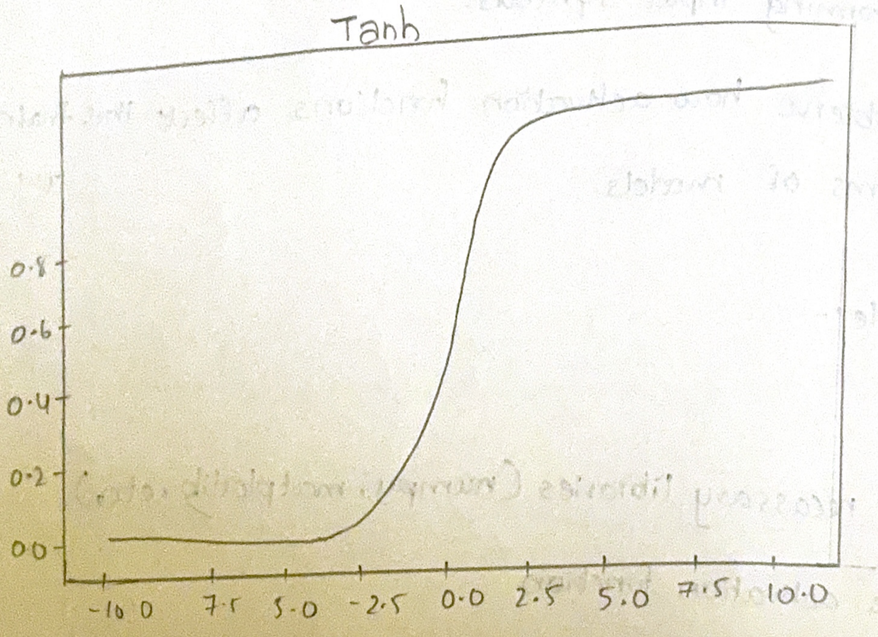
softmax (on vector)



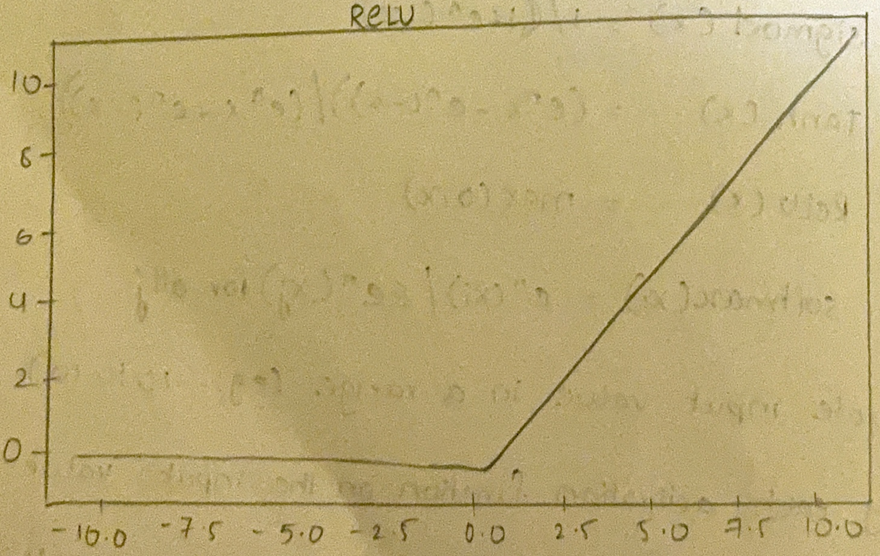
Sigmoid



Tanh



ReLU



final results:-

sigmoid : 93.69%

tanh : 96.18%

relu : 96.00%

```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten

# --------------------------------
# 1. Define activation functions for plotting only (optional)
# --------------------------------
def sigmoid(x): return 1 / (1 + np.exp(-x))
def tanh(x): return np.tanh(x)
def relu(x): return np.maximum(0, x)
def leaky_relu(x): return np.where(x > 0, x, 0.01*x)
def elu(x, alpha=1.0): return np.where(x > 0, x, alpha*(np.exp(x)-1))

x = np.linspace(-6, 6, 400)

# Plot activation function curves
plt.figure(figsize=(12, 8))
plt.subplot(2,3,1); plt.plot(x, sigmoid(x)); plt.title("Sigmoid")
plt.subplot(2,3,2); plt.plot(x, tanh(x)); plt.title("Tanh")
plt.subplot(2,3,3); plt.plot(x, relu(x)); plt.title("ReLU")
plt.subplot(2,3,4); plt.plot(x, leaky_relu(x)); plt.title("Leaky ReLU")
plt.subplot(2,3,5); plt.plot(x, elu(x)); plt.title("ELU")
plt.tight_layout()
plt.show()

# --------------------------------
# 2. Load Dataset
# --------------------------------
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# --------------------------------
# 3. Build model function
# --------------------------------
```

```python
# --------------------------------
def build_model(activation):
    model = Sequential([
        Flatten(input_shape=(28,28)),
        Dense(128, activation=activation),
        Dense(64, activation=activation),
        Dense(10, activation="softmax")  # Softmax in output layer for multi-class classification
    ])
    model.compile(optimizer="adam",
                  loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])
    return model

# --------------------------------
# 4. Train with different activations (ELU removed)
# --------------------------------
activations = ['sigmoid', 'tanh', 'relu']
results = {}

for act in activations:
    print(f"\nTraining with {act} activation...")
    model = build_model(act)
    model.fit(x_train, y_train, epochs=2, batch_size=128, verbose=0)
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    results[act] = acc

# --------------------------------
# 5. Plot comparison chart
# --------------------------------
plt.figure(figsize=(8,5))
plt.bar(results.keys(), results.values(), color=['blue', 'green', 'red'])
plt.title("Accuracy with Different Activation Functions")
plt.ylabel("Accuracy")
plt.show()

# Print results
```

Swayam | New tab | Reduce PPT file size | Convert JPG to PDF. Ima | New tab | Untitled0.ipynb - Colab

colab.research.google.com/drive/1F1iutjGVozUuXgPjePmVwpcNHzCJH-SB

School

Untitled0.ipynb

File Edit View Insert Runtime Tools Help

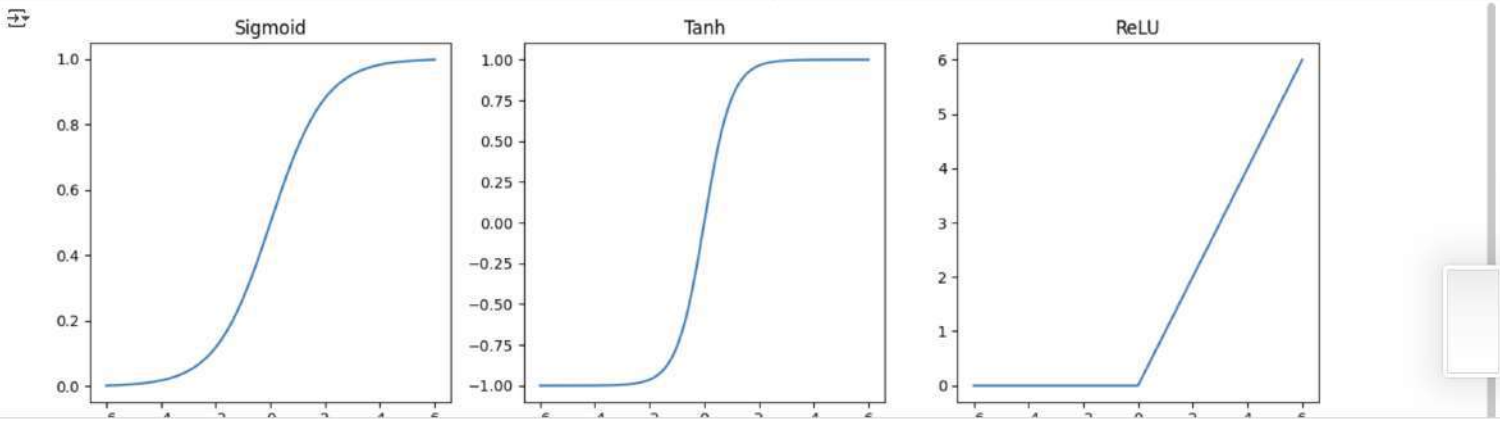
Share Gemini

Commands + Code + Text Run all Connect

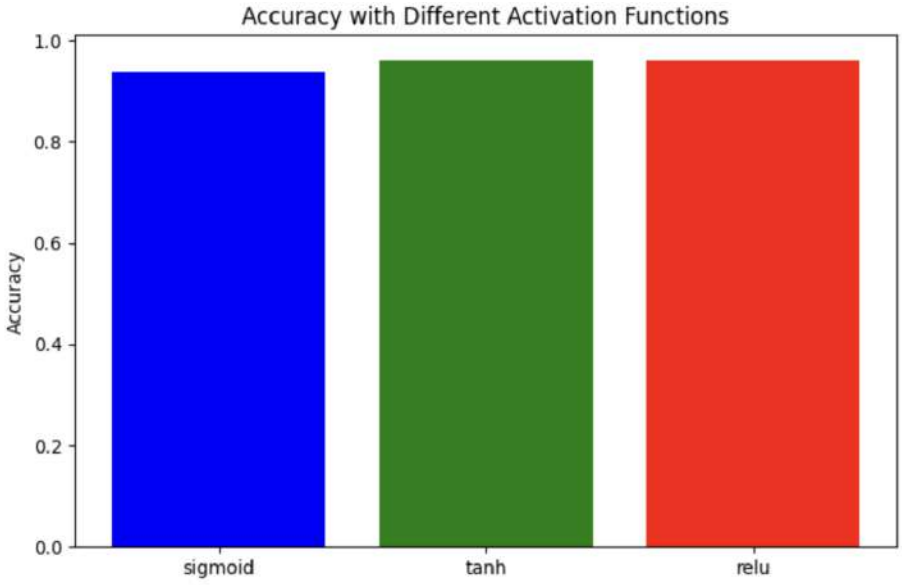
```
[ ]     results[act] = acc

    # ---------------------------------
    # 5. Plot comparison chart
    # ---------------------------------
    plt.figure(figsize=(8,5))
    plt.bar(results.keys(), results.values(), color=['blue', 'green', 'red'])
    plt.title("Accuracy with Different Activation Functions")
    plt.ylabel("Accuracy")
    plt.show()

    # Print results
    print("\nFinal Results:")
    for act, acc in results.items():
        print(f"{act}: {acc*100:.2f}%")
```



Variables Terminal




```
Final Results:
sigmoid: 93.69%
tanh: 96.18%
relu: 96.00%
```